Reason. Plan. Act. Physical AI Workshop

Lab 4.1: Task and motion planning

Guy Azran and Sarah Keren

The Taub Faculty of Computer Science
Technion - Israel Institute of Technology



From “then” to “and”

Guy Azran and
Sarah Keren

Last time: decoupled TAMP. The task planner plans, *then* skills execute the actions, and the robot replans when the world surprises it.

Information flows one way only, so the robot

- kept trying a stack it could not reach
- parked blocks behind the wall



How can the motion layer inform the task planner?

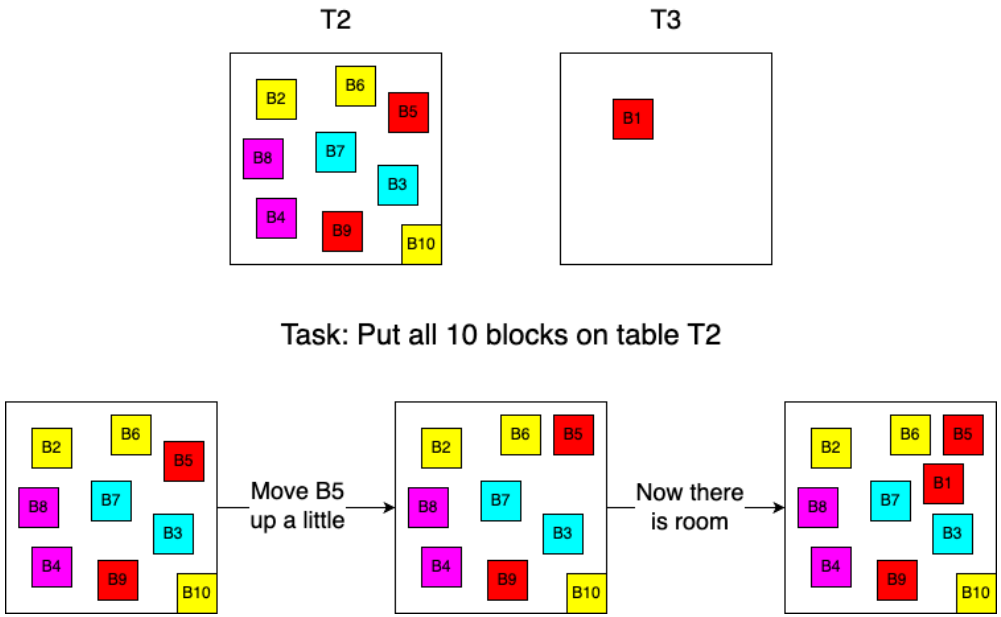
1.1 Task planning	2.1 Motion planning	3.1 State estimation	**4.1 Integrated TAMP**
1.2 Inverse kinematics	2.2 Control	3.2 Decoupled TAMP	4.2 VLAs

Today

Learn	A table with no room: geometry the task planner cannot see
Do	Task 1: the `hasroom` predicate
Do	Task 2: `hasroom` in the state estimator
Learn	The cost of moving, measured by the motion planner
Do	Task 3: the length of a path
Do	Task 4: action costs in the domain
Learn	Simulating plans before moving, and learning from failures
Do	Task 5: lessons from infeasible actions
Do	Bonus: the full table, with room and costs
Learn	What we did not get into

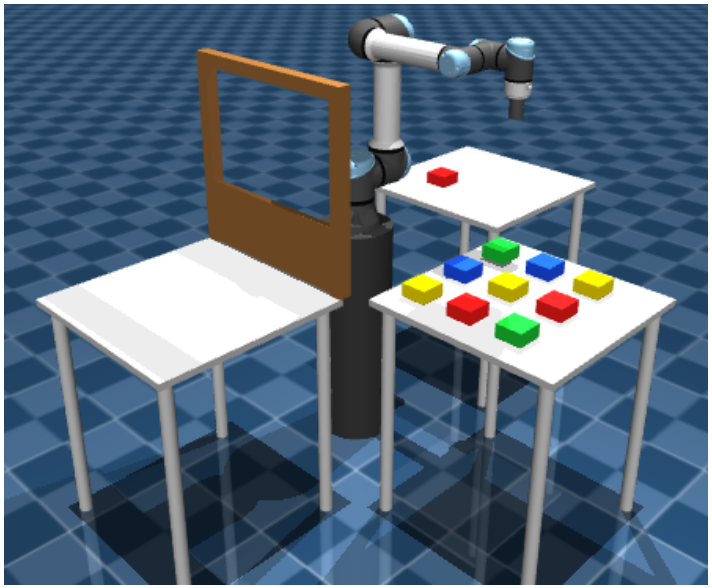
When must the two planners talk?

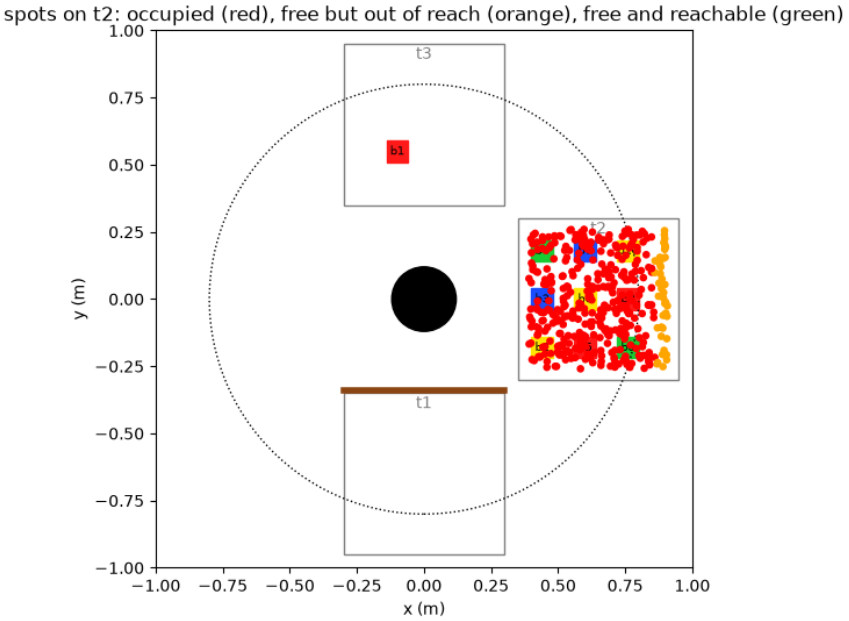
- **Infeasible actions**: applicable in the abstract state, impossible in the world (no room on the table, out of reach)
- **Blind choices**: options that look the same to the task planner but not to the robot (which table, which red block)
- **Wasted effort**: a plan that cannot work, found out one failed skill at a time



A table with no room

Guy Azran and Sarah Keren





Goal: `(ontable b1 t2)`. The sampler finds no free, reachable spot on `t2`.

- The decoupled system plans `put-down(b1, t2)`, the skill fails, and it plans the same action again
- Nothing in the abstract state changes, so neither does the plan

Task 1: teach the planner about room

Guy Azran and Sarah Keren

⚒ **Task 1**: Add the `hasroom` predicate to the domain below. A block can only be put down on a table if `hasroom` holds for that table, and `hasroom` holds for a table right after a block has been picked up from it.

```
  (:action pick-up
     :parameters (?b - block ?t - table)
     :precondition (and (clear ?b) (ontable ?b ?t) (handempty))
     :effect (and (not (ontable ?b ?t)) (not (clear ?b)) (not
(handempty)) (holding ?b)))

  (:action put-down
     :parameters (?b - block ?t - table)
     :precondition (holding ?b)
     :effect (and (not (holding ?b)) (clear ?b) (handempty)
(ontable ?b ?t)))
```

- A new predicate: `(hasroom ?t - table)`
- Which action uses it, and which one makes it true?

Who knows whether there is room?

The domain says how `hasroom` is used and changed. Which tables have room *now* is part of the state.

- It is not about where things are, but about what the robot *can do*
- The component that knows: the **sampler** of the `put-down` skill
- There is room on a table if the sampler finds a spot on it

The motion layer talks to the task planner, **in the task planner's language**.

Can such a fact be wrong? A spot found proves there is room. A thousand misses do not prove there is none.

Task 2: `hasroom` in the state estimator

Guy Azran and
Sarah Keren

⚒ **Task 2**: Write `hasroom(table)`, which returns whether there is room on the table for one more block, using `sample_free_spot(table, block)` (it returns a spot, or `None` after a thousand failed attempts; `block` is a block to ignore, which should be the block that the robot is holding, if any, since it is not on any table).

```python
def hasroom(table):
    """Whether a block can be put down on the table."""
    return True  # your code here
```

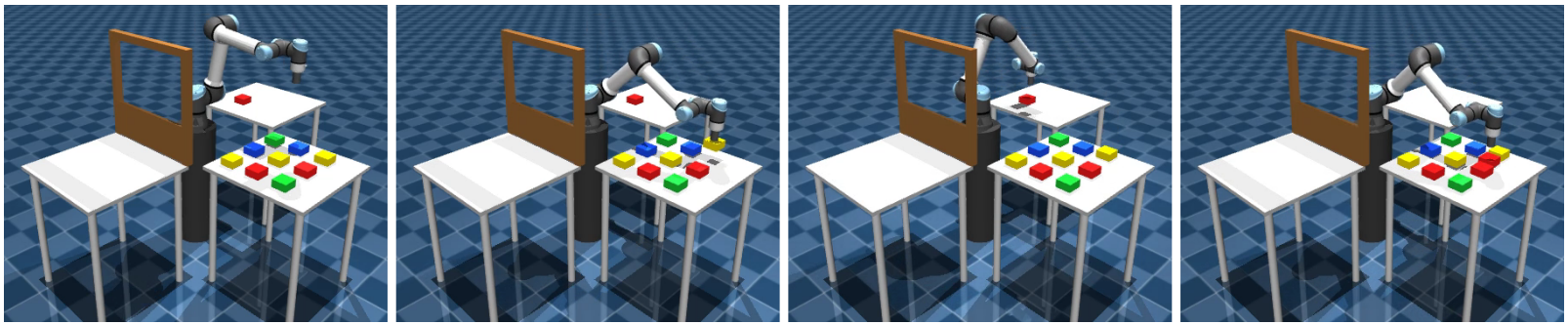
- Ignore the block the robot is holding: it is on no table
- Expect no room on `t2` in the full-table scene

Planning with `hasroom`

Guy Azran and Sarah Keren

```
problem = make_problem(estimate_state(), GOAL,
DOMAIN_WITH_HASROOM)
print(", ".join(map(str, solve(problem))))
```

```
pick-up(b10, t2), put-down(b10, t2), pick-up(b1, t3), put-down(b1,
t2)
```



- It works: a problem the decoupled system could not solve
- But the planner found a **loophole**: `b10` goes back to `t2`, and the robot fidgets until there is room

If every skill works perfectly, how can this still fail?

How much of the world should the model capture?

Gaps between `hasroom` and the truth:

- a truly full table: the robot fidgets forever
- the planner does not know when a table *becomes* full
- room depends on the object, its neighbors and the approach of the arm

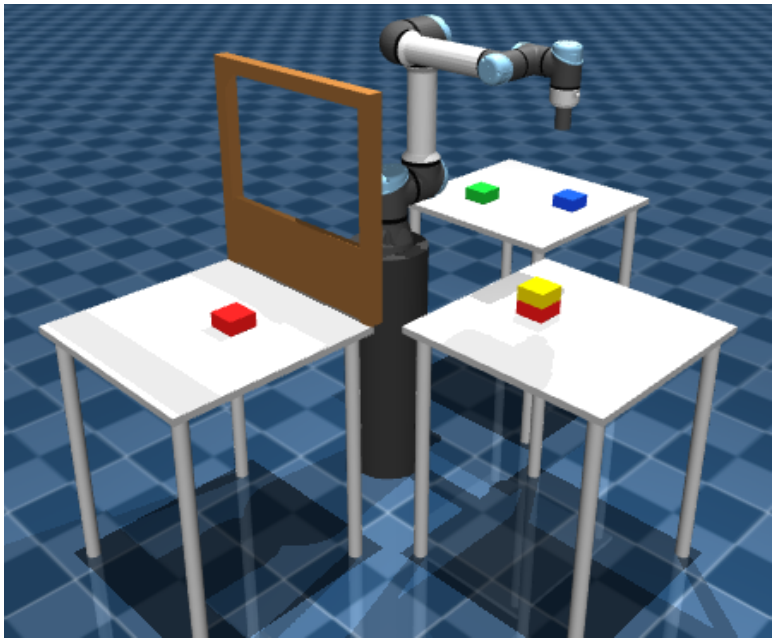
Every refinement (count free places, cells, reachability) makes planning bigger and slower, and none captures the geometry completely.

Our choice: an **optimistic model**, corrected by replanning during execution.

See Kaelbling and Lozano-Pérez, *Hierarchical Task and Motion Planning in the Now*, ICRA 2011.

The cost of moving

Guy Azran and
Sarah Keren



Goal: a red block on `t3`.

- `b1`: clear, behind the window. 2 actions
- `b2`: in front of the robot, under a yellow block. 4 actions

Every action costs 1, so the planner chooses `b1`.

Is `b1` really the cheaper choice? We have a motion planner that plans the very motions the robot will make. Let's ask it.

Task 3: the length of a path

Guy Azran and
Sarah Keren

⚒ **Task 3**: Write `path_length(waypoints)`, which returns the length of a path given as a list of configurations (arrays of six joint angles). The cell uses it to compare the motion from `HOME` to above the red block behind the window with the motion to above the green block on `t3`.

```
def path_length(waypoints):
    """The length of a path in configuration space (radians)."""
    return 0.0  # your code here
```

- Effort of a motion ≈ the length of its path in configuration space (radians)
- Ignores speed, torque and grasping time, but grows with what we care about
- Compare: through the window, and to the table next to the robot

Costs in the task planner's model

Measure once: planned paths from `HOME` to random free spots on each table.

	`t1` (behind the wall)	`t2`	`t3`
mean path length (rad)	9.09	1.94	1.68
`access-cost` (tenths of rad)	91	19	17

- `pick-up` and `put-down` cost the access cost of their table
- `stack` and `unstack` do not say where they happen: a flat `stack-cost` (the median, 19)
- Fast Downward wants integer costs

Task 4: action costs in the domain

Guy Azran and Sarah Keren

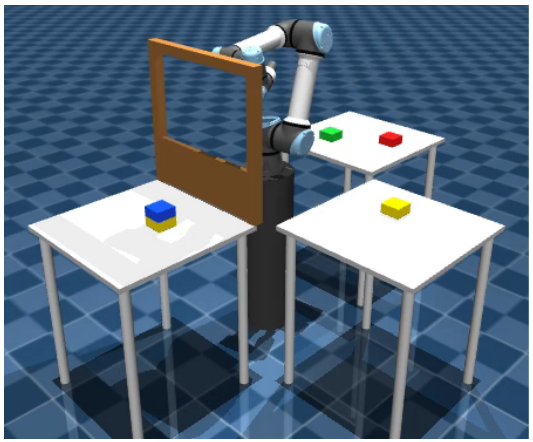
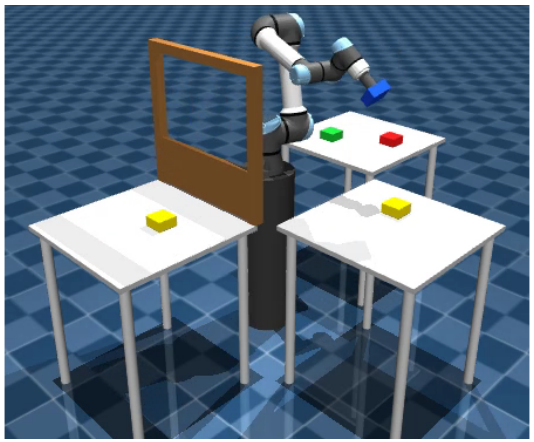
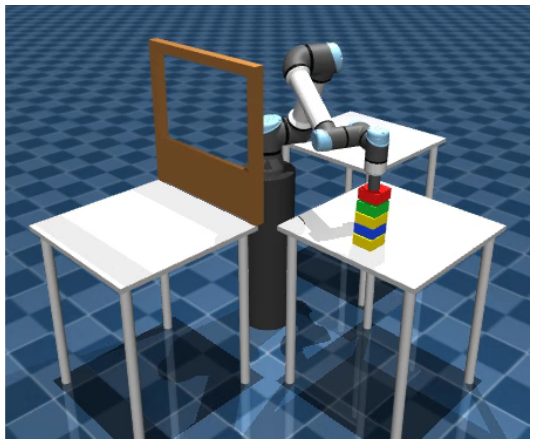
⚒ **Task 4**: Add action costs to the domain (which already has `hasroom`). Declare the numeric functions `(total-cost)`, `(access-cost ?t - table)` and `(stack-cost)`, and add the requirement `:action-costs`. `pick-up` and `put-down` should cost the access cost of their table: add `(increase (total-cost) (access-cost ?t))` to their effects. `stack` and `unstack` do not say where they happen, so charge them a flat `(stack-cost)`. If you need a reminder of the syntax, see the action costs section of tutorial 1.1.

```
  (:action put-down
     :parameters (?b - block ?t - table)
     :precondition (and (holding ?b) (hasroom ?t))
     :effect (and (not (holding ?b)) (clear ?b) (handempty)
(ontable ?b ?t)))
```

- Remember the action costs section of Lab 1.1

Costs change the plan

goal	plan	actions	cost
red block on `t3`	via `b1`	2	108
	via `b2`	4	**72**
tower on `t2`	`b2` parked on `t1`	12	440 (139 s)
	`b2` parked on `t3`	12	**292** (119 s)

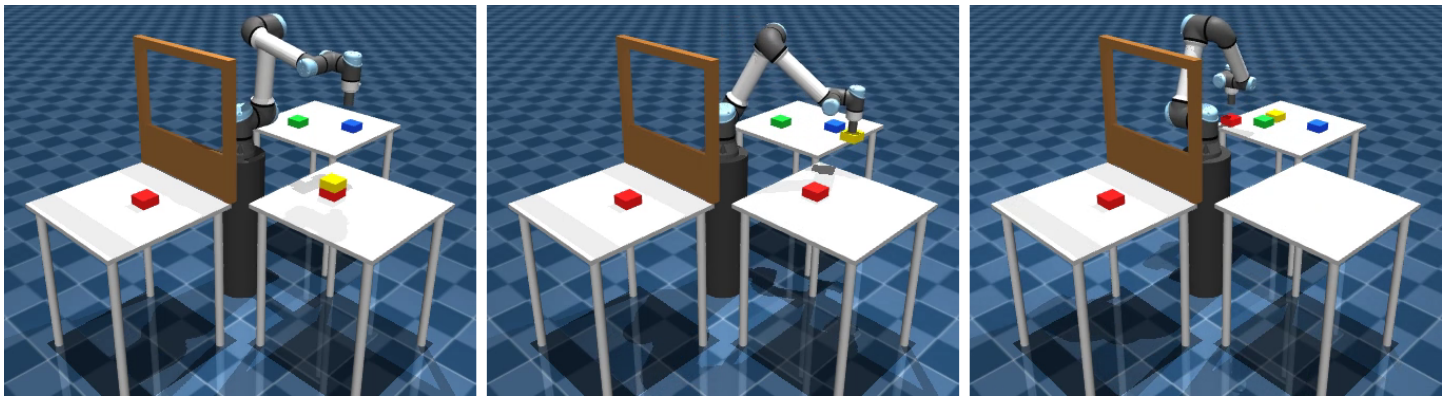
What is wrong with these costs? They are static: one typical trip per table, wherever the arm comes from.

Simulating a plan

Ask the motion layer about the plan itself, in the current world, without moving the robot:

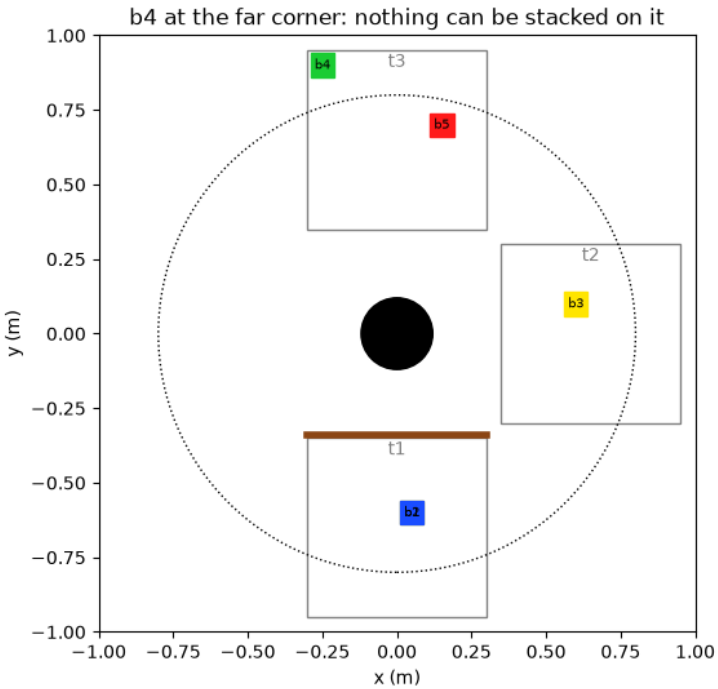
- plan each action's motions, then **teleport** the arm and the block to the outcome
- add up the path lengths, action by action

red block on `t3`	simulated	executed
via **b1** (rejected)	21.0 rad	38 s
via **b2** (chosen)	11.6 rad	24 s



Simulation also finds what cannot be done

Guy Azran and
Sarah Keren



Goal: `(on b5 b4)`, with `b4` out of reach.

`pick-up(b5, t3)`: 2.0 rad
`stack(b5, b4)`: infeasible

Found in 0.1 s of computation, before the robot moved. In Lab 3.2 the robot picked `b5` up and kept trying.

What should the robot do with this knowledge? Turn it into something the task planner understands.

Learning from simulation

Two new facts about what the robot can do with a block *where it is*:

- `(cannot-pick ?b)`
- `(cannot-stack-on ?b)`

- Preconditions like `(not (cannot-pick ?b))` need `:negative-preconditions`
- Optimistic, like `hasroom`: every action is possible until the robot learns otherwise
- Learned **lazily**, only for the actions the planner proposes

propose a plan $\rightarrow$ simulate it $\rightarrow$ learn a lesson $\rightarrow$ plan again

until a plan survives simulation.

Task 5: the lesson of an infeasible action

Guy Azran and Sarah Keren

⚒ **Task 5**: Write `lesson(action)`, which returns the fact (a tuple) that the robot learns when the simulation finds the action infeasible: `("cannot-pick", block)` for `pick-up` or `unstack` of `block`, and `("cannot-stack-on", block)` for `stack` of anything onto `block`. A failed `put-down` teaches nothing that these two facts can say (`hasroom` is there for that): return `None`. Recall from `simulate_action` how to read the name and the arguments of an action.

```
def lesson(action):
    """The fact that the robot learns when the action turns out to be infeasible, or None."""
    return None  # your code here
```

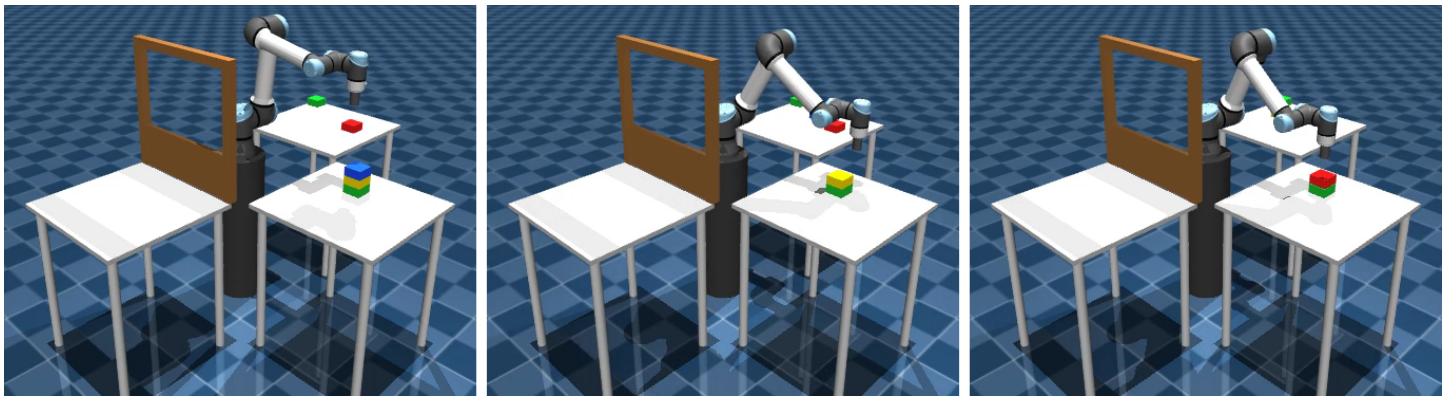
- Read the action's name and arguments as in `simulate_action`

Lessons in action

Guy Azran and
Sarah Keren

Goal: `b5` on *any* green block, an existential goal.

plan 1: stack `b5` on `b4` $\rightarrow$ learn `(cannot-stack-on b4)`
plan 2: move `b4` first $\rightarrow$ learn `(cannot-pick b4)`
plan 3: clear `b3`, 6 actions $\rightarrow$ survives, and is executed



What can go wrong with these lessons? The cause is a guess, the vocabulary is small, and the number of rounds can explode.

Bonus: the full table, with room and costs

Guy Azran and
Sarah Keren

⚒ **Bonus task**: Solve the problem of the full table once more (`FULL_LAYOUT` , `GOAL`), in closed loop, this time with the domain that has both `hasroom` and costs. Before you run it: what do you expect the robot to do with the block it picks up from `t2` , and why?

```
# your code here
```

- Before you run it: where will the block from `t2` go, and why?
- Did we close the loophole of `hasroom`?

Get to work!

Summary

In this tutorial we

- gave the task planner a geometric fact, `hasroom`, computed by the sampler
- saw how loosely such facts fit the world, and relied on replanning for the rest
- measured action costs with the motion planner, and stopped parking blocks behind the wall
- simulated plans before moving, to choose the cheapest one and to find what cannot be done
- learned lessons from infeasible actions, and planned until a plan survived simulation

What comes next

What we did not get into: partial observability, multiple agents, navigation, richer geometry, and learning the skills, samplers, predicates and costs. (survey: Garrett et al., *Integrated Task and Motion Planning*, 2021)

Almost everything in our robot is written by hand: the domain, the skills, the samplers, the controller, the costs.

Next, Lab 4.2: **end-to-end learning**. One model, trained on large amounts of data, maps what the robot sees and what it is asked to do directly to actions: **vision-language-action models (VLAs)**.

1.1 Task planning	2.1 Motion planning	3.1 State estimation	4.1 Integrated TAMP
1.2 Inverse kinematics	2.2 Control	3.2 Decoupled TAMP	**4.2 VLAs**